OPIPE-03: Sampling-Aware Metrics

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 3 of 6 | **Created:** March 2026 | **Last Updated:** 08/27/2026

Extracting Accurate Metrics from Sampled Trace Data

When distributed tracing uses sampling (head sampling, tail sampling, or adaptive sampling), only a fraction of spans are stored. Extracting metrics from these sampled spans requires special handling — a naive `count()` on sampled data gives you a fraction of reality, not the truth.

This notebook explains what sampling-aware metrics are, why they matter, and how to configure OpenPipeline to extract accurate RED metrics (Rate, Errors, Duration) from sampled span data.

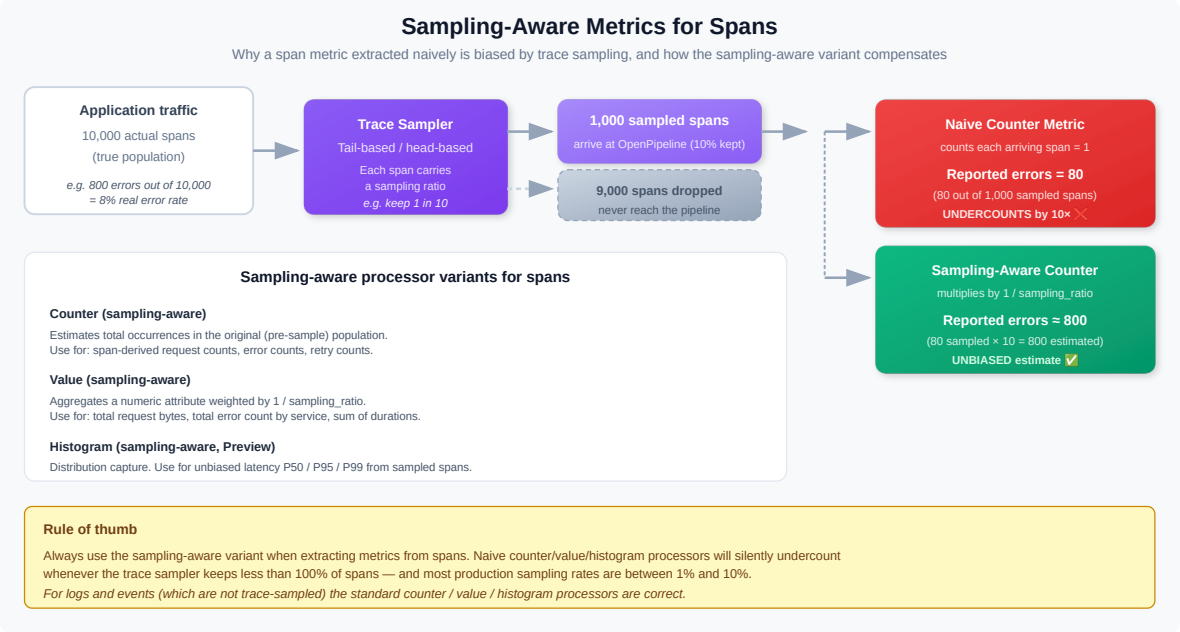
For log-derived metrics (which are not affected by sampling), see **OPMIG-07: Metric & Event Extraction**.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and distributed tracing
Permissions	`storage:spans:read`, `storage:metrics:read`, `openpipeline:configurations:write`
Data	Active span data from instrumented services
Recommended	**OPIPE-02** (span processing) and **SPANS-01** (fundamentals)

1. The Sampling Problem



Why Sampling Exists

Distributed tracing generates enormous volumes of data. A single user request can produce dozens of spans across multiple services. At scale — thousands of requests per second — storing every span is prohibitively expensive.

Sampling solves this by retaining a representative subset:

Sampling Type	How It Works	Trade-off
Head sampling	Decision at trace start (e.g., keep 10% of traces)	Simple but misses rare errors
Tail sampling	Decision after trace completes (keep errors, slow traces)	Captures important traces, needs collector buffer
Adaptive sampling	Adjusts rate based on traffic volume	Balances cost and coverage dynamically

The Metric Accuracy Problem

When you extract metrics from sampled spans, the raw numbers are wrong:

Reality	10% Head Sampling	Naive Metric
10,000 requests/min	1,000 spans stored	`count() = 1,000` (10x under-reported)
500 errors/min	~50 error spans stored	`countIf(error) = 50` (10x under-reported)
Avg latency 200ms	~1,000 spans sampled	`avg(duration) = ~200ms` (approximately correct)

Key insight: Counts and rates are severely affected by sampling. Averages and percentiles are approximately correct (the sample is statistically representative). This distinction drives how sampling-aware metrics work.

```
// Check: Is your environment using span sampling?
//
// The field is `dt.system.sampling_ratio` (stable). `sampling.ratio` and
// `sampling.probability` DO NOT EXIST — neither has a row in the semantic
// dictionary, so filtering on either returns null with no error and gives you
// no way to tell "not sampled" from "wrong field name". Verified 08/27/2026:
// dt.system.sampling_ratio was populated on all 642,584 spans in a 2h window,
// while sampling.threshold was null on every one of them.
// `supportability.atm_sampling_ratio` (experimental) carries the Adaptive
// Traffic Management ratio specifically, on 99.8% of those spans.
fetch spans, from:-1h
| fieldsKeep trace.id, span.kind, service.name
| summarize span_count = count(), by:{service.name}
| sort span_count desc
| limit 10
```

```
// Is sampling actually in effect? A ratio of 1 means every span is kept.
fetch spans, from:-1h
| summarize spans = count(), by:{dt.system.sampling_ratio}
| sort spans desc
```

2. What Makes a Metric Sampling-Aware

A **sampling-aware metric** adjusts its calculation based on the sampling rate that was applied to the source data. Instead of counting raw spans, it compensates for the sampling ratio to produce values that reflect the actual traffic.

How It Works

Each sampled span carries metadata about the sampling decision — typically a sampling ratio or probability. When OpenPipeline extracts a metric, it can use this information:

Metric Type	Standard Extraction	Sampling-Aware Extraction
Request count	`count()` → 1,000	`count()` weighted by 1/sampling_ratio → 10,000
Error count	`countIf(error)` → 50	`countIf(error)` weighted → 500
Error rate	50/1,000 = 5%	500/10,000 = 5% (same — ratio cancels out)
Avg duration	`avg(duration)` → 200ms	`avg(duration)` → 200ms (same — sample is representative)
P95 duration	`percentile(duration, 95)` → 800ms	`percentile(duration, 95)` → ~800ms (approximately same)

When Sampling-Awareness Matters

Metric	Sampling-Aware Needed?	Why
Request rate (throughput)	**Yes**	Absolute counts are under-reported
Error rate (ratio)	No	Ratio of sampled errors to sampled total is representative
Error count (absolute)	**Yes**	Absolute counts are under-reported

Metric	Sampling-Aware Needed?	Why
Average latency	No	Sample mean approximates population mean
P95/P99 latency	Partially	Approximation degrades at extreme percentiles with aggressive sampling
Throughput for SLO	**Yes**	SLO calculations need accurate request volume

3. RED Metrics from Spans

The **RED method** (Rate, Errors, Duration) is the standard framework for service-level metrics. OpenPipeline can extract all three from span data.

Rate (Request Throughput)

Measures how many requests a service handles per unit of time.

- **Source**: Server spans (`span.kind == "server"`)
- **Aggregation**: Count, weighted by sampling ratio
- **Dimensions**: `service.name` , `http.route` , `k8s.namespace.name`
- **Sampling-aware**: **Yes** — must compensate for sampling to get true request rate

Errors (Failure Rate)

Measures the proportion of requests that fail.

- **Source**: Server spans where `http.response.status_code >= 500` or `span.status_code == "error"`
- **Aggregation**: Count of errors / count of total requests
- **Dimensions**: `service.name` , `http.route` , `http.response.status_code`
- **Sampling-aware**: Error rate (ratio) is naturally sampling-tolerant; absolute error count is not

> **The failure field is `span.status_code` , and it is lowercase.** `otel.status_code` does not exist in Grail — it has no row in `dt.semantic_dictionary.fields` — and the value is `"error"` , not the SDK's uppercase `ERROR` . Both mistakes return **zero rows without an error**, which on a sampling notebook is doubly dangerous: a zero error count reads as "sampling is discarding my errors" and sends you tuning the sampler instead of fixing the field name.
>
> A third trap compounds it: **`span.status_code` is null on successful spans** (`"ok"` is written vanishingly rarely). So `countIf(span.status_code != "error")` counts almost nothing — derive successes as **`total - errors`** . A sampling-compensation factor applied to a zero is still zero, so this error survives every downstream correction.

Duration (Latency)

Measures how long requests take to complete.

- **Source**: Server spans, `duration` field
- **Aggregation**: Average, P50, P95, P99
- **Dimensions**: `service.name` , `http.route`
- **Sampling-aware**: No — duration distributions from sampled data are statistically representative

```
// RED: Request rate by service (from stored spans)
fetch spans, from:-1h
| filter span.kind == "server"
| makeTimeseries request_count = count(), by:{service.name}, interval:5m
```

```
// RED: Error rate by service
// Counts both HTTP 5xx and spans the instrumentation marked failed.
// span.status_code is the real field (not otel.status_code) and its value is
// lowercase "error"; it is null on successful spans, so successes = total - errors.
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    total = count(),
    errors = countIf(http.response.status_code &gt;= 500 or span.status_code == "error")
  }, by:{service.name}
| fieldsAdd successes = total - errors
| fieldsAdd error_rate_pct = round(100.0 * errors / total, decimals: 2)
| sort error_rate_pct desc
```

```
// RED: Duration percentiles by service
fetch spans, from:-1h
| filter span.kind == "server"
| summarize p50 = percentile(duration, 50),
    p95 = percentile(duration, 95),
    p99 = percentile(duration, 99),
    by:{service.name}
| sort p95 desc
| limit 15
```

4. Configuring Metric Extraction in OpenPipeline

Metric extraction from spans is configured in the **Spans scope** of OpenPipeline, under the **Extraction** processing stage.

Configuration Steps

1. **Navigate** to OpenPipeline > Spans > select your pipeline > Extraction
2. **Add metric extraction rule** with:
   - **Metric key**: The name of the metric to create (e.g., `span.request_count`)
   - **Aggregation**: `count`, `sum`, `min`, `max`, `avg`, `value` (for gauges)
   - **Condition**: Which spans to extract from (e.g., `span.kind == "server"`)
   - **Dimensions**: Fields to carry as metric dimensions (e.g., `service.name`, `http.route`)

Example: RED Metric Extraction Rules

Metric Key	Aggregation	Condition	Dimensions	Sampling-Aware
`span.request_count`	count	`span.kind == "server"`	`service.name`, `http.route`	Yes
`span.error_count`	count	`span.kind == "server"` AND `http.response.status_code >= 500`	`service.name`, `http.route`, `http.response.status_code`	Yes

Metric Key	Aggregation	Condition	Dimensions	Sampling-Aware
`span.duration`	value	`span.kind == "server"`	`service.name`, `http.route`	No (duration is representative)

Dimension Selection: Less Is More

Every dimension you add multiplies the number of metric data points (cardinality). Choose dimensions carefully:

Dimension	Cardinality Impact	Include?
`service.name`	Low (tens)	Always
`http.route`	Medium (hundreds)	Usually
`http.response.status_code`	Low (5-10 values)	For error metrics
`k8s.namespace.name`	Low (tens)	If multi-tenant
`span.name`	High (thousands)	Rarely — too granular
`trace.id`	Extreme (unique per trace)	**Never** — destroys metric performance

Cardinality management is covered in depth in **OPIPE-04: Cardinality Management**.

5. Comparing Standard vs. Sampling-Aware Metrics

After configuring metric extraction, validate that sampling-aware metrics produce accurate results by comparing them against known baselines.

```
// Compare: Raw span count vs. built-in service request metric
// If sampling is active, raw span count will be lower than the metric
fetch spans, from:-1h
| filter span.kind == "server"
| summarize raw_span_count = count(), by:{service.name}
| sort raw_span_count desc
| limit 10
```

```
// Built-in service request count metric (not affected by span sampling)
timeseries request_count = sum(dt.service.request.count), from:-1h, by:{dt.entity.service}
| fieldsAdd total_requests = arraySum(request_count)
| sort total_requests desc
| limit 10
```

6. Cardinality Considerations

Metric extraction creates new metric data points. The total number of data points (cardinality) is the product of all dimension values:

```
Cardinality = unique(service.name) x unique(http.route) x unique(status_code) x ...
```

Example: 50 services x 200 routes x 5 status codes = **50,000 time series**

Cardinality Guardrails

Guideline	Threshold
Acceptable	< 10,000 time series per metric
Caution	10,000 - 100,000 time series
Danger	> 100,000 time series (metric may be dropped or throttled)

Reducing Cardinality Before Extraction

Use OpenPipeline processing stages **before** extraction to normalize high-cardinality fields:

- Replace URL paths with route patterns: `/users/12345/orders` → `/users/{id}/orders`
- Group status codes: `200`, `201`, `204` → `2xx`
- Remove query parameters from `http.url`

Full cardinality management strategies are covered in **OPIPE-04: Cardinality Management**.

Summary

In this notebook you learned:

- **The sampling problem** — Counts are under-reported on sampled data; averages and ratios are approximately correct
- **Sampling-aware metrics** — Compensate for sampling ratio to produce accurate throughput and error counts
- **RED metrics from spans** — Rate (request count), Errors (failure rate), Duration (latency percentiles)
- **Metric extraction configuration** — Rules, conditions, dimensions, and aggregation types
- **Cardinality awareness** — Every dimension multiplies data points; choose dimensions deliberately

---

Next Steps

Continue to **OPIPE-04: Cardinality Management** for strategies to control dimension explosion across all OpenPipeline scopes — span attributes, log fields, and metric dimensions.

References

- Extract metrics from spans and distributed traces (DT docs)
- Data storage and retention for Distributed Tracing (DT docs)
- The RED Method (Grafana)
- Metric limits and cardinality (DT docs)
